

LP3 (ISR) – Assignment 2 – Single Pass Algorithm (RollNo: 43221)

Input Code:

```
import java.io.*;
import java.util.*;

public class SinglePassAlgorithm {
    public static void main(String[] args) throws IOException {
        BufferedReader stdInput = new BufferedReader(new
InputStreamReader(System.in));
        System.out.println("Enter the number of Documents");
        int noOfDocuments = Integer.parseInt(stdInput.readLine());
        System.out.println("Enter the number of Tokens");
        int noOfTokens = Integer.parseInt(stdInput.readLine());
        System.out.println("Enter the threshold");
        float threshold = Float.parseFloat(stdInput.readLine());
        System.out.println("Enter the Document Token Matrix");
        int[][] input = new int[noOfDocuments][noOfTokens];
        for (int i = 0; i < noOfDocuments; ++i) {
            for (int j = 0; j < noOfTokens; ++j) {
                System.out.println("Enter(" + i + "," + j + ")");
                input[i][j] = Integer.parseInt(stdInput.readLine());
            }
        }
        singlePassAlgorithm(noOfDocuments, noOfTokens, threshold, input);
    }

    private static void singlePassAlgorithm(int noOfDocuments, int
noOfTokens, float threshold, int[][] input) {
        int[][] cluster = new int[noOfDocuments][noOfDocuments + 1];
        ArrayList<Float[]> clusterRepresentative = new ArrayList<>();
        cluster[0][0] = 1;
        cluster[0][1] = 0;
        int noOfClusters = 1;
        Float[] temp = convertIntArrayToFloatArray(input[0]);
        clusterRepresentative.add(temp);
        for (int i = 1; i < noOfDocuments; ++i) {
            float max = -1;
            int clusterId = -1;
            for (int j = 0; j < noOfClusters; ++j) {
                float similarity =
calculateSimilarity(convertIntArrayToFloatArray(input[i]),
clusterRepresentative.get(j));
                if (similarity > threshold) {
                    if (similarity > max) {
                        max = similarity;
                        clusterId = j;
                    }
                }
            }
            if (max == -1) {
                cluster[noOfClusters][0] = 1;
                cluster[noOfClusters][1] = i;
                noOfClusters++;
            }
            clusterRepresentative.add(convertIntArrayToFloatArray(input[i]));
        } else {
            cluster[clusterId][0] += 1;
        }
    }
}
```

```

        int index = cluster[clusterId][0];
        cluster[clusterId][index] = i;
        clusterRepresentative.set(clusterId,
            calculateClusterRepresentative(cluster[clusterId],
                input, noOfTokens));
    }
}

for (int i = 0; i < noOfClusters; ++i) {
    System.out.println("\nCluster " + i + ":");
    System.out.print("Documents: ");
    for (int j = 1; j <= cluster[i][0]; ++j) {
        System.out.print(cluster[i][j] + " ");
    }
    System.out.println();
    System.out.println("Cluster Representative: " +
        arrayToString(clusterRepresentative.get(i)));
}

private static String arrayToString(Float[] array) {
    StringBuilder sb = new StringBuilder();
    sb.append("[");
    for (int i = 0; i < array.length; i++) {
        sb.append(array[i]);
        if (i != array.length - 1) {
            sb.append(", ");
        }
    }
    sb.append("]");
    return sb.toString();
}

private static Float[] convertIntArrayToFloatArray(int[] input) {
    int size = input.length;
    Float[] answer = new Float[size];
    for (int i = 0; i < input.length; ++i) {
        answer[i] = (float) input[i];
    }
    return answer;
}

private static float calculateSimilarity(Float[] a, Float[] b) {
    float answer = 0;
    for (int i = 0; i < a.length; ++i) {
        answer += a[i] * b[i];
    }
    return answer;
}

private static Float[] calculateClusterRepresentative(int[] cluster,
    int[][] input,
int noOfTokens) {
    Float[] answer = new Float[noOfTokens];
    for (int i = 0; i < noOfTokens; ++i) {
        answer[i] = 0f;
    }
    for (int i = 1; i <= cluster[0]; ++i) {
        for (int j = 0; j < noOfTokens; ++j) {
            answer[j] += input[cluster[i]][j];
        }
    }
}

```

```

        for (int i = 0; i < noOfTokens; ++i) {
            answer[i] /= cluster[0];
        }
        return answer;
    }
}

```

Output:

"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.1.4\lib\idea_rt.jar=57915:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.1.4\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath F:\BE\LP3\GroupA_Assignment_2_SinglePass\out\production\GroupA_Assignment_2_SinglePass SinglePassAlgorithm

Enter the number of Documents

4

Enter the number of Tokens

4

Enter the threshold

0.5

Enter the Document Token Matrix

Enter(0,0)

1

Enter(0,1)

0

Enter(0,2)

0

Enter(0,3)

0

Enter(1,0)

0

Enter(1,1)

1

Enter(1,2)

0

Enter(1,3)

0

Enter(2,0)

0

Enter(2,1)

0

Enter(2,2)

1

Enter(2,3)

0

Enter(3,0)

0

Enter(3,1)

0

Enter(3,2)

0

Enter(3,3)

1

Cluster 0:

Documents: 0

Cluster Representative: [1.0, 0.0, 0.0, 0.0]

Cluster 1:

Documents: 1

Cluster Representative: [0.0, 1.0, 0.0, 0.0]

Cluster 2:

Documents: 2

Cluster Representative: [0.0, 0.0, 1.0, 0.0]

Cluster 3:

Documents: 3

Cluster Representative: [0.0, 0.0, 0.0, 1.0]

Process finished with exit code 0